

Exame de Paradigmas de Programação — Época de Recurso / Modelo 2

|||

|---|---|

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso / Modelo 2
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1

1,5 val

Explique a diferença entre tipos de dados primitivos e tipos de referência em Java. Aborde os conceitos de *Wrapper Classes*, *Autoboxing* e *Unboxing*, fundamentando como a memória é alocada e gerida para cada um destes tipos (Stack vs. Heap). Ilustre com um exemplo prático.

Pergunta 2

1,5 val

No contexto do design Orientado a Objetos (SOLID), explique a relevância do Princípio da Responsabilidade Única (*Single Responsibility Principle* - SRP) e do Princípio Aberto/Fechado (*Open/Closed Principle* - OCP). Apresente um exemplo simples de código em Java que viole o OCP e mostre como o refatorar para cumprir este princípio.

Pergunta 3

1,5 val

Descreva detalhadamente o funcionamento dos modificadores de acesso em Java (`private`, `package-private` / `default`, `protected` e `public`). Explique de que modo a escolha adequada da visibilidade de atributos e métodos reforça o conceito de Encapsulamento e protege a integridade dos objetos.

Pergunta 4

1,5 val

Explique a utilização e o impacto da palavra-chave `final` em Java quando aplicada a:

1. Variáveis (primitivas e de referência);
2. Métodos;
3. Classes.

Discuta de que forma a declaração de métodos ou classes como `final` afeta os mecanismos de Herança e Polimorfismo.

Parte 2

1. Considere as seguintes interfaces `AidBox` e `SmartAidBox`. A interface `AidBox` define o contrato base para caixas de suprimentos da instituição. A interface `SmartAidBox` especializa `AidBox`, adicionando capacidade de monitorização da bateria do seu módulo IoT e deteção automática da necessidade de manutenção.

```
public interface AidBox {  
    String getCode();  
    Container[] getContainers();  
    double getLatitude();  
    double getLongitude();  
}
```

```
public interface SmartAidBox extends AidBox {  
    double getBatteryLevel();  
    boolean needsMaintenance();  
    boolean equals(Object obj);  
}
```

Pergunta 1a

3 val

Implemente a interface `SmartAidBox` numa classe denominada `SmartAidBoxImpl`. A classe deve possuir um **estado** (`OPERATIONAL`, `MAINTENANCE_REQUIRED`) que inicia como `OPERATIONAL`.

Regras de Implementação:

- O método `needsMaintenance()` deve retornar `true` se o nível da bateria (`batteryLevel`) for inferior a 15.0% **OU** se pelo menos um dos seus contentores tiver a última medição (`getLastMeasurement().getValue()`) superior a 95.0% da sua capacidade máxima (`getCapacity()`).

- Se o método `needsMaintenance()` retornar `true`, o estado da caixa deve passar para `MAINTENANCE_REQUIRED`.
- Para a implementação do método `equals`, considere que **duas instâncias** de `SmartAidBox` são iguais se possuírem o mesmo código (devolvido por `getCode()`).

Pergunta 1b

2 val

Desenvolva o código necessário para testar a classe `SmartAidBoxImpl` num método `main`. O teste deve instanciar uma caixa inteligente, adicionar-lhe contentores e testar as condições que provocam a alteração de estado para manutenção e a verificação de igualdade entre caixas.

2. Considere a existência de uma classe `AlertServiceImpl` que implementa a interface `AlertService`.

```
public interface AlertService {  
    SmartAidBox[] getBoxesNeedingMaintenance(IIInstitution inst);  
    boolean sendAlert(SmartAidBox box, AlertSystem system);  
    int processAllAlerts(IIInstitution inst, AlertSystem system);  
}
```

Pergunta 2a

4 val

Na classe `AlertServiceImpl`, implemente os seguintes métodos auxiliares:

```
SmartAidBox[] getBoxesNeedingMaintenance(IIInstitution inst);
```

- Este método deve percorrer o array de `AidBox` devolvido por `inst.getAidBoxes()`.
- Deve filtrar apenas as caixas que sejam instâncias de `SmartAidBox` e cujo método `needsMaintenance()` devolva `true`.
- Deve devolver um array perfeitamente dimensionado contendo apenas as `SmartAidBox` identificadas (sem posições nulas ou elementos irrelevantes).

```
boolean sendAlert(SmartAidBox box, AlertSystem system);
```

- O método deve tentar enviar uma notificação de manutenção invocando `system.notifyMaintenance(box)`.
- Se a chamada a `system.notifyMaintenance` decorrer com sucesso, o método deve retornar `true`.
- Caso essa chamada lance uma exceção do tipo `AlertException`, o método deve capturar a exceção e retornar `false`.

Pergunta 2b

5 val

Na classe `AlertServiceImpl`, implemente o método principal `processAllAlerts`:

```
int processAllAlerts(IIInstitution inst, AlertSystem system);
```

Regras a considerar:

- Utilize o método `getBoxesNeedingMaintenance` desenvolvido na alínea anterior para obter a lista de caixas inteligentes que necessitam de intervenção.
- Para cada uma dessas caixas, utilize o método `sendAlert` para enviar o alerta ao `AlertSystem`.
- Mantenha uma contagem de quantos alertas foram efetivamente enviados com sucesso.
- No final do processamento, devolva o número total de alertas enviados com sucesso.

Excertos de Código Fornecidos

```
public interface IIInstitution {
    AidBox[] getAidBoxes();
}

public interface AidBox {
    String getCode();
    Container[] getContainers();
}

public interface Container {
    double getCapacity();
    Measurement getLastMeasurement();
}

public interface Measurement {
    double getValue();
}

public interface AlertSystem {
    void notifyMaintenance(SmartAidBox box) throws AlertException;
}
```